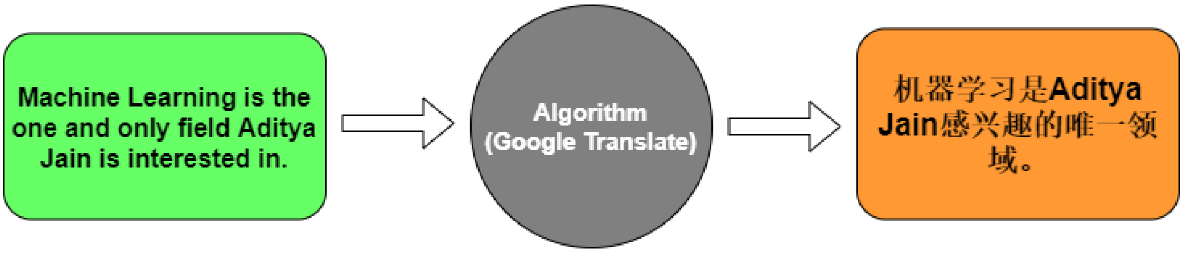
Name: Mohaiya aldossary
Student ID: 2230004579

Introduction to Machine Translation (MT):

Machine Translation is a subfield of AI and linguistics that develops systems capable of translating text from one language to another. The primary goal of machine translation is to enable effective communication between people who speak different languages by automating the process of language translation.


Machine Learning is the one and only field Aditya Jain is interested in.
Algorithm (Google Translate)
机器学习是Aditya Jain感兴趣的唯一领域。

Challenges in Machine Translation:

- Ambiguity

Ex. I went to the bank.

- Idiomatic Expressions

Ex. It's raining cats and dogs.

- Low-Resource Languages

Some languages have very limited data.

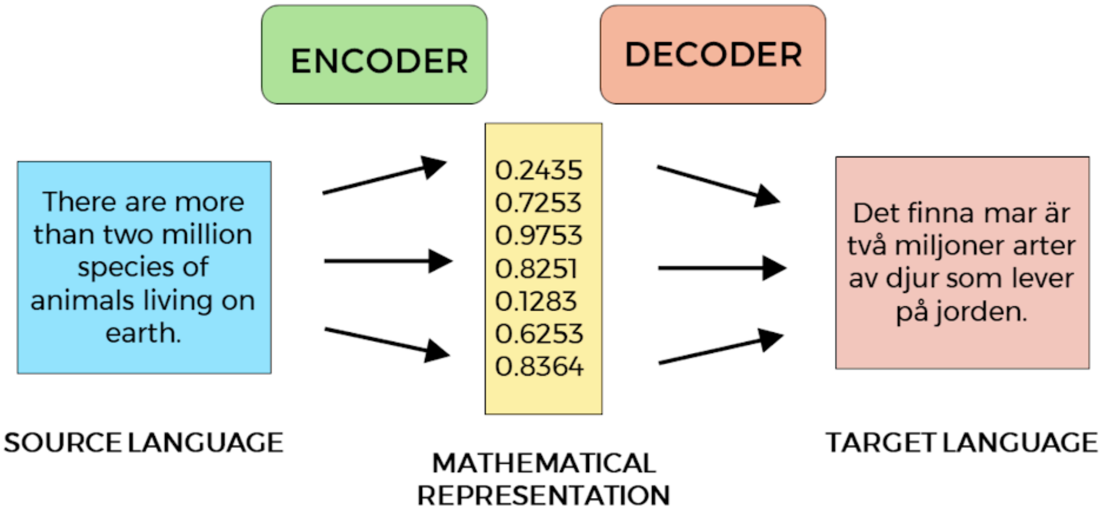
Applications of Machine Translation:

- Global Business and E-commerce -Cross-Cultural Communication
- Multilingual Content Creation
- Language Learning

Neural Machine Translation:

Neural Machine Translation is a data-driven approach where a single large neural network learns to represent both the source language and the target language, and to directly map between them. Instead of using manually defined rules or phrase tables, it learns translation patterns automatically from large datasets of bilingual text.

Key Components of Neural Machine Translation:


ENCODER
DECODER
There are more than two million species of animals living on earth.
0.2435
0.7253
0.9753
0.8251
0.1283
0.6253
0.8364
Det finna mar är två miljoner arter av djur som lever på jorden.
SOURCE LANGUAGE
MATHEMATICAL REPRESENTATION
TARGET LANGUAGE

In [1]:

```
%pip -q install "tensorflow==2.16.1" "numpy>=1.26,<2" "pandas>=2.1,<4" scikit-learn nltk
```

```
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 23.2.1 -> 26.2.2
[notice] To update, run: C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\.lab_work\venv10\Scripts\python.exe -m pip install --
upgrade pip
```

In [2]:

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
import numpy as np

# Example bilingual dataset
english_sentences = ["Hello.", "How are you?", "This is a machine translation example."]
french_sentences = ["Bonjour.", "Comment ça va?", "Ceci est un exemple de traduction automatique."]

# Tokenize the english sentences
tokenizer_eng = Tokenizer()
tokenizer_eng.fit_on_texts(english_sentences) # Tokenize words and assign an integer ID to each unique word
eng_sequences = tokenizer_eng.texts_to_sequences(english_sentences) # Returns a list of tokens replaced by their index.
Note: words index will start from 1 because 0 is reserved by default for padding (not assigned to any word).
print(f"eng_sequences = {eng_sequences}")
eng_word_index = tokenizer_eng.word_index # Returns the dictionary created by the tokenizer dict {word: index}.
print(f"eng_word_index = {eng_word_index}")
```

```
eng_sequences = [[1], [2, 3, 4], [5, 6, 7, 8, 9, 10]]
eng_word_index = {'hello': 1, 'how': 2, 'are': 3, 'you': 4, 'this': 5, 'is': 6, 'a': 7, 'machine': 8, 'translation': 9,
'example': 10}
```

In [3]:

```python
# Tokenize the french sentences
tokenizer_frn = Tokenizer()
tokenizer_frn.fit_on_texts(french_sentences)
frn_sequences = tokenizer_frn.texts_to_sequences(french_sentences)

print(f"frn_sequences = {frn_sequences}")
frn_word_index = tokenizer_frn.word_index
print(f"frn_word_index = {frn_word_index}")
```

```
frn_sequences = [[1], [2, 3, 4], [5, 6, 7, 8, 9, 10, 11]]
frn_word_index = {'bonjour': 1, 'comment': 2, 'ça': 3, 'va': 4, 'ceci': 5, 'est': 6, 'un': 7, 'exemple': 8, 'de': 9, 'traduction': 10, 'automatique': 11}
```

In [4]:

```python
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Padding sequences
eng_padded = pad_sequences(eng_sequences) # makes all sequences the same length by padding with zeros at the beginning (default)
print(f"eng_padded = {eng_padded}")
```

```
eng_padded = [[ 0  0  0  0  0  1]
 [ 0  0  0  2  3  4]
 [ 5  6  7  8  9 10]]
```

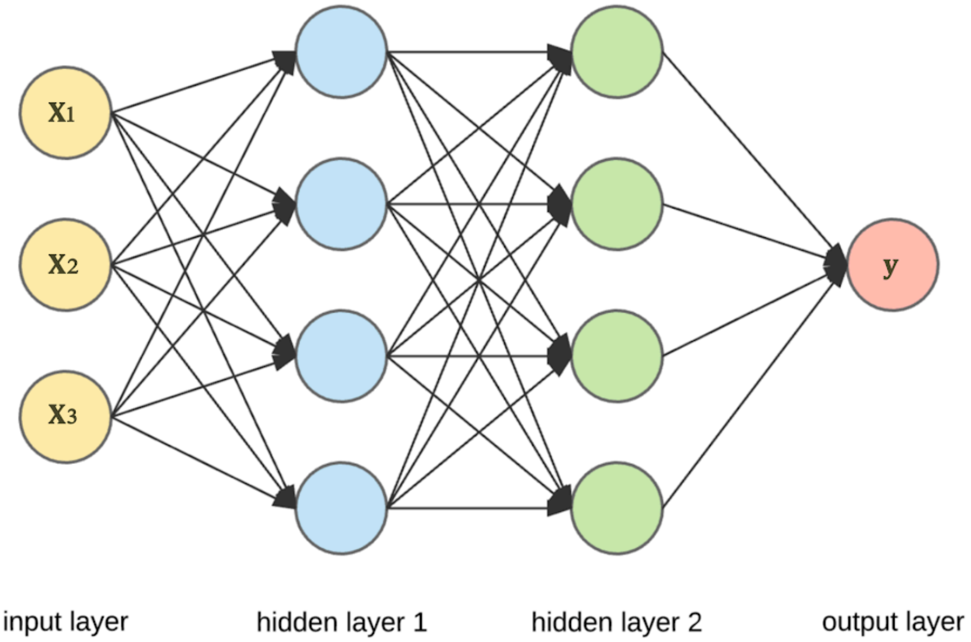
In [5]:

```python
frn_padded = pad_sequences(frn_sequences)
frn_padded
```

Out[5]:

```
array([[ 0,  0,  0,  0,  0,  0,  1],
       [ 0,  0,  0,  0,  2,  3,  4],
       [ 5,  6,  7,  8,  9, 10, 11]])
```

LSTM Model



In [6]:

```python
from sklearn.model_selection import train_test_split

# Train-test split
eng_train, eng_test, frn_train, frn_test = train_test_split(eng_padded, frn_padded, test_size=0.2, random_state=42)

# Define the model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(eng_word_index) + 1, output_dim=64),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.RepeatVector(frn_padded.shape[1]), #length of french sentence
    tf.keras.layers.LSTM(64, return_sequences=True),
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(len(frn_word_index) + 1, activation='softmax')) #convert to
probabilities for each frn word
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(eng_train, np.expand_dims(frn_train, -1), epochs=50, batch_size=1, validation_split=0.2)

# Evaluate the model
loss, accuracy = model.evaluate(eng_test, np.expand_dims(frn_test, -1))
print(f'Test Accuracy: {accuracy}')
```

```
Epoch 1/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.0000e+00 - loss: 2.4851
�������������������������������������������������������������������������
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step - accuracy: 0.0000e+00 - loss: 2.4851 - val_accuracy: 0.0000e+00 - val_loss: 2.4872
Epoch 2/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.5714 - loss: 2.4746
�����������������������������������������������������������������������
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.5714 - loss: 2.4746 - val_accuracy: 0.0000e+00 - val_loss: 2.4898
Epoch 3/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 2.4638
�����������������������������������������������������������������������
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - accuracy: 0.5714 - loss: 2.4638 - val_accuracy: 0.0000e+00 - val_loss: 2.4925
Epoch 4/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5714 - loss: 2.4522
�����������������������������������������������������������������������
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step - accuracy: 0.5714 - loss: 2.4522 - val_accuracy: 0.0000e+00 - val_loss: 2.4956
Epoch 5/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5714 - loss: 2.4394
�����������������������������������������������������������������������
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 2.4394 - val_accuracy: 0.0000e+00 - val_loss: 2.4990
Epoch 6/50
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 2.4249

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - accuracy: 0.5714 - loss: 2.4249 - val_accuracy: 0.0000e+00 - val_loss: 2.5030
Epoch 7/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.5714 - loss: 2.4081

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 72ms/step - accuracy: 0.5714 - loss: 2.4081 - val_accuracy: 0.0000e+00 - val_loss: 2.5075
Epoch 8/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 2.3886

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.5714 - loss: 2.3886 - val_accuracy: 0.0000e+00 - val_loss: 2.5128
Epoch 9/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 2.3657

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 2.3657 - val_accuracy: 0.0000e+00 - val_loss: 2.5190
Epoch 10/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 2.3385

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 2.3385 - val_accuracy: 0.0000e+00 - val_loss: 2.5262
Epoch 11/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5714 - loss: 2.3063

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step - accuracy: 0.5714 - loss: 2.3063 - val_accuracy: 0.0000e+00 - val_loss: 2.5348
Epoch 12/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5714 - loss: 2.2681

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 0.5714 - loss: 2.2681 - val_accuracy: 0.0000e+00 - val_loss: 2.5450
Epoch 13/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 2.2227

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 2.2227 - val_accuracy: 0.0000e+00 - val_loss: 2.5571
Epoch 14/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5714 - loss: 2.1692

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - accuracy: 0.5714 - loss: 2.1692 - val_accuracy: 0.0000e+00 - val_loss: 2.5719
Epoch 15/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.5714 - loss: 2.1066

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - accuracy: 0.5714 - loss: 2.1066 - val_accuracy: 0.0000e+00 - val_loss: 2.5899
Epoch 16/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5714 - loss: 2.0347

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 73ms/step - accuracy: 0.5714 - loss: 2.0347 - val_accuracy: 0.0000e+00 - val_loss: 2.6121
Epoch 17/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.5714 - loss: 1.9545

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step - accuracy: 0.5714 - loss: 1.9545 - val_accuracy: 0.0000e+00 - val_loss: 2.6396
Epoch 18/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5714 - loss: 1.8689

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 77ms/step - accuracy: 0.5714 - loss: 1.8689 - val_accuracy: 0.0000e+00 - val_loss: 2.6738
Epoch 19/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.7825

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step - accuracy: 0.5714 - loss: 1.7825 - val_accuracy: 0.0000e+00 - val_loss: 2.7160
Epoch 20/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.5714 - loss: 1.7002

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step - accuracy: 0.5714 - loss: 1.7002 - val_accuracy: 0.0000e+00 - val_loss: 2.7675
Epoch 21/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.6238

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 1.6238 - val_accuracy: 0.0000e+00 - val_loss: 2.8291
Epoch 22/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.5515

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 1.5515 - val_accuracy: 0.0000e+00 - val_loss: 2.9013
Epoch 23/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.4807

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.5714 - loss: 1.4807 - val_accuracy: 0.0000e+00 - val_loss: 2.9846
Epoch 24/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.4102

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.5714 - loss: 1.4102 - val_accuracy: 0.0000e+00 - val_loss: 3.0786
Epoch 25/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.3400

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.5714 - loss: 1.3400 - val_accuracy: 0.0000e+00 - val_loss: 3.1823
Epoch 26/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5714 - loss: 1.2709

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 0.5714 - loss: 1.2709 - val_accuracy: 0.0000e+00 - val_loss: 3.2938
Epoch 27/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.5714 - loss: 1.2035
```

```
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 93ms/step - accuracy: 0.5714 - loss: 1.2035 - val_accuracy: 0.0000e+00 - val_loss: 3.4112
Epoch 28/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5714 - loss: 1.1389
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - accuracy: 0.5714 - loss: 1.1389 - val_accuracy: 0.0000e+00 - val_loss: 3.5322
Epoch 29/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.5714 - loss: 1.0788
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 65ms/step - accuracy: 0.5714 - loss: 1.0788 - val_accuracy: 0.0000e+00 - val_loss: 3.6547
Epoch 30/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.5714 - loss: 1.0246
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 77ms/step - accuracy: 0.5714 - loss: 1.0246 - val_accuracy: 0.0000e+00 - val_loss: 3.7768
Epoch 31/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5714 - loss: 0.9775
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.5714 - loss: 0.9775 - val_accuracy: 0.0000e+00 - val_loss: 3.8969
Epoch 32/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.7143 - loss: 0.9373
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - accuracy: 0.7143 - loss: 0.9373 - val_accuracy: 0.0000e+00 - val_loss: 4.0137
Epoch 33/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7143 - loss: 0.9029
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.7143 - loss: 0.9029 - val_accuracy: 0.0000e+00 - val_loss: 4.1268
Epoch 34/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.8722
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.8722 - val_accuracy: 0.0000e+00 - val_loss: 4.2365
Epoch 35/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.8433
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 0.7143 - loss: 0.8433 - val_accuracy: 0.0000e+00 - val_loss: 4.3434
Epoch 36/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.8143
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.8143 - val_accuracy: 0.0000e+00 - val_loss: 4.4477
Epoch 37/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.7859
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.7859 - val_accuracy: 0.0000e+00 - val_loss: 4.5492
Epoch 38/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7143 - loss: 0.7616
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.7616 - val_accuracy: 0.0000e+00 - val_loss: 4.6470
Epoch 39/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.7443
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step - accuracy: 0.7143 - loss: 0.7443 - val_accuracy: 0.0000e+00 - val_loss: 4.7392
Epoch 40/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.7143 - loss: 0.7287
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - accuracy: 0.7143 - loss: 0.7287 - val_accuracy: 0.0000e+00 - val_loss: 4.8260
Epoch 41/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.7109
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - accuracy: 0.7143 - loss: 0.7109 - val_accuracy: 0.0000e+00 - val_loss: 4.9077
Epoch 42/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7143 - loss: 0.6923
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.7143 - loss: 0.6923 - val_accuracy: 0.0000e+00 - val_loss: 4.9846
Epoch 43/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.7143 - loss: 0.6740
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 75ms/step - accuracy: 0.7143 - loss: 0.6740 - val_accuracy: 0.0000e+00 - val_loss: 5.0562
Epoch 44/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7143 - loss: 0.6573
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step - accuracy: 0.7143 - loss: 0.6573 - val_accuracy: 0.0000e+00 - val_loss: 5.1221
Epoch 45/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.6409
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step - accuracy: 0.7143 - loss: 0.6409 - val_accuracy: 0.0000e+00 - val_loss: 5.1824
Epoch 46/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7143 - loss: 0.6259
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.6259 - val_accuracy: 0.0000e+00 - val_loss: 5.2372
Epoch 47/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7143 - loss: 0.6140
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.6140 - val_accuracy: 0.0000e+00 - val_loss: 5.2869
Epoch 48/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.7143 - loss: 0.6028
```

```
▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.7143 - loss: 0.6028 - val_accuracy: 0.0000e+00 - val_loss: 5.3326
Epoch 49/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7143 - loss: 0.5899
▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - accuracy: 0.7143 - loss: 0.5899 - val_accuracy: 0.0000e+00 - val_loss: 5.3751
Epoch 50/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.7143 - loss: 0.5773
▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step - accuracy: 0.7143 - loss: 0.5773 - val_accuracy: 0.0000e+00 - val_loss: 5.4149
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 434ms/step - accuracy: 0.7143 - loss: 1.7077
▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 436ms/step - accuracy: 0.7143 - loss: 1.7077
Test Accuracy: 0.7142857313156128
```

In [7]:

```python
# Inference
def translate_sentence(sentence, model, eng_tokenizer, frn_tokenizer):
    sequence = eng_tokenizer.texts_to_sequences([sentence])
    padded_sequence = pad_sequences(sequence, maxlen=eng_padded.shape[1])
    predicted_sequence = model.predict(padded_sequence) #outputs probabilities for each timestep (french padded sentence) and each unique word in the target vocabulary.
    translated_sequence = np.argmax(predicted_sequence, axis=-1) #Returns the index (word ID) with the largest value at each timestep.
    translated_sentence = frn_tokenizer.sequences_to_texts(translated_sequence)
    return translated_sentence[0]

# Example Inference
input_sentence = "How are you?"
translated_sentence = translate_sentence(input_sentence, model, tokenizer_eng, tokenizer_frn)
print(f'Input: {input_sentence}\nTranslation: {translated_sentence}')
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 281ms/step
▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 281ms/step
Input: How are you?
Translation: va va
```

Task 1: Understanding the Model Layers

Answer these simple questions in your own words:

1. What does the **Embedding** layer do to a word index?
2. What is the role of the **Encoder LSTM**?
3. What does the **Decoder LSTM** do?
4. Why do we use `softmax` in the final layer?

1. The Embedding layer changes each word index into a trainable vector of numbers that represents the word.
2. The Encoder LSTM reads the English sequence and summarizes its information in a context vector.
3. The Decoder LSTM uses the repeated context vector to produce a sequence of hidden states for the French words.
4. Softmax turns the final scores into probabilities over the French vocabulary, so we can select the most likely word at each position.

Task 2:

Use Case: Language Translation (English-French)

A global e-commerce company wants to expand its reach to French-speaking customers and provide a seamless shopping experience. The company needs a language translation solution to automatically translate product listings, customer reviews, and other relevant content from English to French.

link: https://www.kaggle.com/datasets/devicharith/language-translation-englishfrench

In [8]:

```
from pathlib import Path
from urllib.request import urlopen
from zipfile import ZipFile
from io import BytesIO
import pandas as pd

data_path = Path("data/eng_-french.csv")
if not data_path.exists():
    data_path.parent.mkdir(exist_ok=True)
    url = "https://www.kaggle.com/api/v1/datasets/download/devicharith/language-translation-englishfrench"
    with urlopen(url, timeout=180) as response:
        with ZipFile(BytesIO(response.read())) as archive:
            data_path.write_bytes(archive.read("eng_-french.csv"))

pairs = pd.read_csv(data_path).dropna()
pairs.columns = ["English", "French"]
print("Dataset pairs:", len(pairs))
display(pairs.head())
```

```
Dataset pairs: 175621
```

	English	French
0	Hi.	Salut!
1	Run!	Cours !
2	Run!	Courez !
3	Who?	Qui ?
4	Wow!	Ça alors !

In [9]:

```python
import re
import unicodedata

tf.keras.utils.set_random_seed(42)

def normalize(text):
    text = unicodedata.normalize("NFKC", text).lower()
    text = re.sub(r"[^\w\s']", " ", text)
    return " ".join(text.split())

pairs = pairs.map(normalize)
pairs = pairs[(pairs.English.str.split().str.len().between(1, 5)) &
              (pairs.French.str.split().str.len().between(1, 7))]
pairs = pairs.drop_duplicates("English").sample(n=10000, random_state=42)
train_pairs, test_pairs = train_test_split(pairs, test_size=0.1, random_state=42)
train_pairs, val_pairs = train_test_split(train_pairs, test_size=0.1, random_state=42)

source_tokenizer = Tokenizer(oov_token="<unk>")
target_tokenizer = Tokenizer(oov_token="<unk>")
source_tokenizer.fit_on_texts(train_pairs.English)
target_tokenizer.fit_on_texts(train_pairs.French)

def encode(frame):
    source = pad_sequences(source_tokenizer.texts_to_sequences(frame.English),
                           maxlen=5, padding="post")
    target = pad_sequences(target_tokenizer.texts_to_sequences(frame.French),
                           maxlen=7, padding="post")
    return source, target

x_train, y_train = encode(train_pairs)
x_val, y_val = encode(val_pairs)
x_test, y_test = encode(test_pairs)
print("Training pairs:", len(train_pairs))
print("Validation pairs:", len(val_pairs))
print("Test pairs:", len(test_pairs))
print("English vocabulary:", len(source_tokenizer.word_index) + 1)
print("French vocabulary:", len(target_tokenizer.word_index) + 1)
```

```
Training pairs: 8100
Validation pairs: 900
Test pairs: 1000
English vocabulary: 3915
French vocabulary: 5344
```

In [10]:

```python
translation_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(5,)),
    tf.keras.layers.Embedding(len(source_tokenizer.word_index) + 1, 64, mask_zero=True),
    tf.keras.layers.LSTM(128),
    tf.keras.layers.RepeatVector(7),
    tf.keras.layers.LSTM(128, return_sequences=True),
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(
        len(target_tokenizer.word_index) + 1, activation="softmax"))
])
translation_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
translation_model.summary()
history = translation_model.fit(
    x_train, y_train, validation_data=(x_val, y_val),
    epochs=30, batch_size=64, verbose=2,
    callbacks=[tf.keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=5, restore_best_weights=True)]
)
```

```
Model: "sequential_1"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ embedding_1 (Embedding)         │ (None, 5, 64)          │       250,560 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ lstm_2 (LSTM)                   │ (None, 128)            │        98,816 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ repeat_vector_1 (RepeatVector)  │ (None, 7, 128)         │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ lstm_3 (LSTM)                   │ (None, 7, 128)         │       131,584 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ time_distributed_1              │ (None, 7, 5344)        │       689,376 │
│ (TimeDistributed)               │                        │               │
└─────────────────────────────────┴────────────────────────┴───────────────┘

 Total params: 1,170,336 (4.46 MB)

 Trainable params: 1,170,336 (4.46 MB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/30
127/127 - 6s - 46ms/step - loss: 5.8162 - val_loss: 5.1282
Epoch 2/30
127/127 - 3s - 22ms/step - loss: 4.9604 - val_loss: 5.0517
Epoch 3/30
127/127 - 3s - 22ms/step - loss: 4.8152 - val_loss: 4.9363
Epoch 4/30
127/127 - 3s - 23ms/step - loss: 4.6937 - val_loss: 4.8540
Epoch 5/30
127/127 - 3s - 22ms/step - loss: 4.5985 - val_loss: 4.7904
Epoch 6/30
127/127 - 3s - 22ms/step - loss: 4.5089 - val_loss: 4.7383
Epoch 7/30
127/127 - 3s - 22ms/step - loss: 4.4402 - val_loss: 4.7108
Epoch 8/30
127/127 - 3s - 22ms/step - loss: 4.3890 - val_loss: 4.7029
Epoch 9/30
127/127 - 3s - 22ms/step - loss: 4.3524 - val_loss: 4.6687
Epoch 10/30
127/127 - 3s - 22ms/step - loss: 4.3079 - val_loss: 4.6590
Epoch 11/30
127/127 - 3s - 22ms/step - loss: 4.2636 - val_loss: 4.6398
Epoch 12/30
127/127 - 3s - 23ms/step - loss: 4.2219 - val_loss: 4.6226
Epoch 13/30
127/127 - 3s - 26ms/step - loss: 4.1847 - val_loss: 4.6018
Epoch 14/30
127/127 - 3s - 26ms/step - loss: 4.1551 - val_loss: 4.5821
Epoch 15/30
127/127 - 3s - 26ms/step - loss: 4.1059 - val_loss: 4.5475
Epoch 16/30
127/127 - 3s - 25ms/step - loss: 4.0469 - val_loss: 4.5220
Epoch 17/30
127/127 - 3s - 26ms/step - loss: 3.9946 - val_loss: 4.5090
Epoch 18/30
127/127 - 3s - 26ms/step - loss: 3.9459 - val_loss: 4.4918
Epoch 19/30
127/127 - 3s - 25ms/step - loss: 3.9101 - val_loss: 4.4914
Epoch 20/30
127/127 - 3s - 26ms/step - loss: 3.8757 - val_loss: 4.4893
Epoch 21/30
127/127 - 3s - 25ms/step - loss: 3.8569 - val_loss: 4.4674
```

```
Epoch 22/30
127/127 - 3s - 25ms/step - loss: 3.7947 - val_loss: 4.4453
Epoch 23/30
127/127 - 3s - 25ms/step - loss: 3.7381 - val_loss: 4.4318
Epoch 24/30
127/127 - 3s - 26ms/step - loss: 3.6903 - val_loss: 4.4263
Epoch 25/30
127/127 - 3s - 25ms/step - loss: 3.6436 - val_loss: 4.4121
Epoch 26/30
127/127 - 3s - 25ms/step - loss: 3.5995 - val_loss: 4.4111
Epoch 27/30
127/127 - 3s - 26ms/step - loss: 3.5646 - val_loss: 4.4137
Epoch 28/30
127/127 - 3s - 25ms/step - loss: 3.5476 - val_loss: 4.3616
Epoch 29/30
127/127 - 3s - 25ms/step - loss: 3.5046 - val_loss: 4.3366
Epoch 30/30
127/127 - 3s - 26ms/step - loss: 3.4329 - val_loss: 4.3352
```

In [11]:

```python
from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction

test_loss = translation_model.evaluate(x_test, y_test, verbose=0)
predicted_ids = translation_model.predict(x_test, batch_size=64, verbose=0).argmax(axis=-1)
token_accuracy = (predicted_ids[y_test != 0] == y_test[y_test != 0]).mean()
predictions = target_tokenizer.sequences_to_texts([
    [int(word_id) for word_id in row if word_id != 0] for row in predicted_ids
])
references = [[text.split()] for text in test_pairs.French]
bleu = corpus_bleu(references, [text.split() for text in predictions],
                   smoothing_function=SmoothingFunction().method1)
print(f"Test loss: {test_loss:.4f}")
print(f"Test token accuracy (excluding padding): {token_accuracy:.4f}")
print(f"Test BLEU (0-1, one reference per sentence): {bleu:.4f}")
display(pd.DataFrame({"English": test_pairs.English.to_numpy()[:5],
                      "Reference": test_pairs.French.to_numpy()[:5],
                      "Prediction": predictions[:5]}))

def translate_product(text):
    encoded = pad_sequences(source_tokenizer.texts_to_sequences([normalize(text)]),
                            maxlen=5, padding="post")
    ids = translation_model.predict(encoded, verbose=0).argmax(axis=-1)
    return target_tokenizer.sequences_to_texts([
        [int(word_id) for word_id in ids[0] if word_id != 0]
    ])[0]

for text in ["This is a good product.", "The price is too high.", "I like this shirt."]:
    print(f"English: {text}\nFrench: {translate_product(text)}\n")
```

```
Test loss: 4.3176
Test token accuracy (excluding padding): 0.1671
Test BLEU (0-1, one reference per sentence): 0.0050
```

	English	Reference	Prediction
0	i won't close the door	je ne fermerai pas la porte	je ne pas pas pas pas
1	the antelope ran away	l'antilope s'enfuit	ils sont sont
2	i spoke with tom's assistant	j'ai parlé à l'assistant de tom	je ce que de de de
3	no one is coming	personne ne vient	personne ne
4	i'm not racist	je ne suis pas raciste	je ne pas pas pas

```
English: This is a good product.
French: c'est une une

English: The price is too high.
French: c'est est est
English: I like this shirt.
French: j'aime les
```